

Matthew Bernardino
Professor Anshul
ECPE 136
12 February 2026

Lab Assignment 3 - ALU

Objective:

- The objective of this lab assignment is to learn how to design and make an ALU (Arithmetic Logic Unit). There is a 3 bit control variable 0-7 that will select which math operation will be used on inputs A and B. In particular we are doing 8 bit inputs A and B to add and subtract numbers. When we add and subtract we can add in a carry in that adds an additional 1 simulating this happening. We also are able to do other operations like AND, OR, shift left, shift right, rotate left, and rotate right. Each of these operations will have test values to verify specific inputs to outputs.

Procedures:

1.0 - Create ALU_8bit.sv

- Creating a top level script to set up the inputs and outputs. Particularly I used a case statement to make each math operation be associated with its opcode from 000 to 111. Each particular number will be associated with an opcode like 000 being Add and 001 being Subtract. We use the “always_comb begin” logic because we have combinational logic and we want to re-evaluate every time our inputs change.

2.0 - Create ALU_8bit_tb.sv

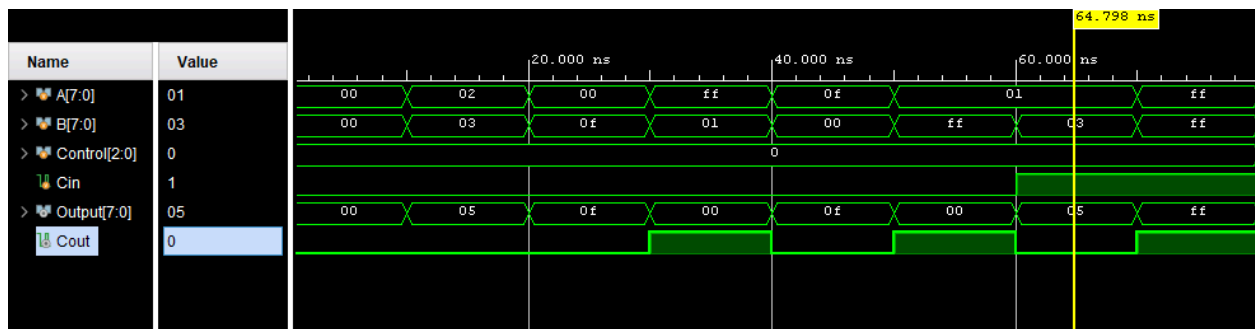
- Create a test bench to test various values for each opcode. Within the testbench we must set up all inputs and outputs. We then instantiate the inputs and outputs from the main file. During testing for each we use the control variable to select the operation we want based on the opcode. Then we pass our inputs A,B, and Cin to reach a specific test we want to do. Only the Add and Sub operations need Cin. Also the operations OR, AND, shl, shr, rol, ror all are only able to use one input. Within our tests we use input A as the main input for testing.

Design Description:

Control	Instruction	Operation
000	Add	Output = A+B+Cin; Cout = carry-ou
001	Sub	Output = A-B-Cin; Cout = borrow-out
010	OR	Output = A B
011	AND	Output = A&B
100	Shl	Output = {A[6:0], 1'b0} (logical shift left by 1)
101	Shr	Output = {1'b0, A[7:1]} (logical shift right by 1)
110	Rol	Output = {A[6:0], A[7]} (rotate left by 1)
111	Ror	Output = {A[0], A[7:1]} (rotate right by 1)

- When designing this 8-bit ALU, behavioral modeling was the main choice. I used a “always_comb” block because we are constantly updating the inputs and this block constantly updates for every signal change. Since we have multiple changes when considering the operations, I used a case statement that uses input 3 bit control to determine which operation is being selected. As seen in the above opcode table layout. When control is updated the inputs will be performing the math associated with the specific 3 bit control. Each case statement has a control for the math operation and the math equation for how it should function.

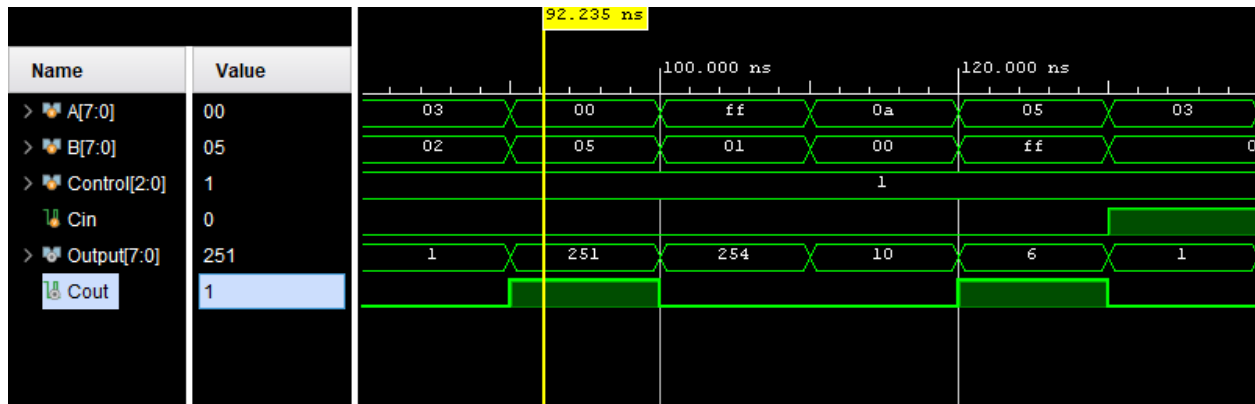
Observations & Results:



ADD Waveform

ADD Observation:

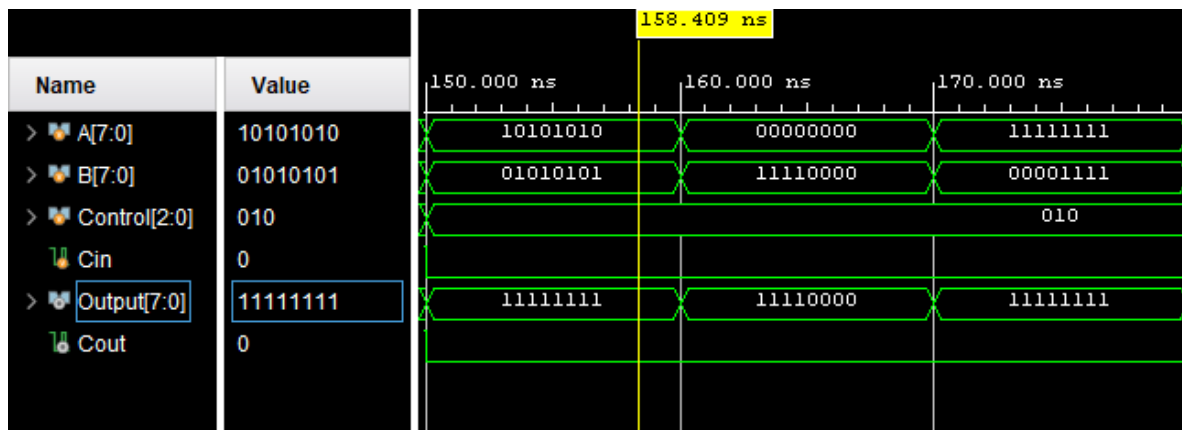
- Within the waveform we can see the inputs A=1, B=3, Cin=1. The formula will add all three and so $1+3=4$ and $4+1=5$. We can see the correct output via the waveform and for other input changes. When overflow happens like adding 255 with any number greater than 0, Cout is produced and becomes a 9 bit total output of 8 bit sum and 1 bit cout.



SUB Waveform

SUB Observation:

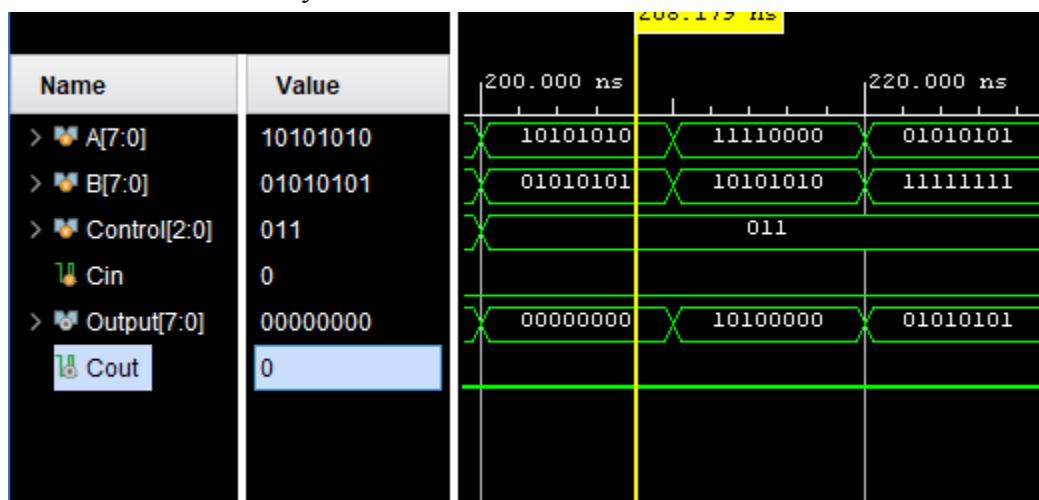
- This waveform shows the subtraction opcode. When we subtract we subtract all inputs $A-B-Cin$. When a negative result happens like in $0-5$, a $Cout=1$ is produced as the negative bit and the subtraction wraps around from 256 showing output 251.



OR Waveform

OR Observation:

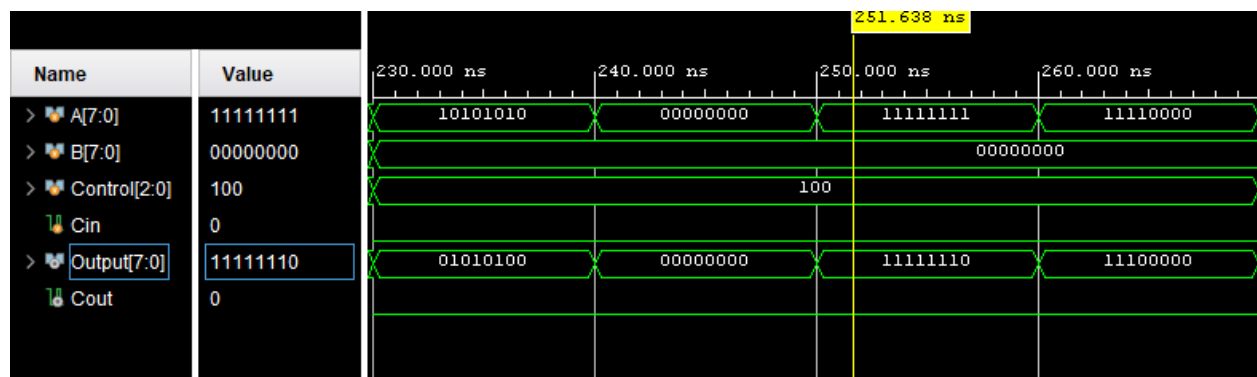
- OR logic is just like the gates in which if there is a 1 then it will accept it. Within the examples we can see the alternating A and B inputs and when the OR logic computes, the values combine in an 8-bit 1 answer. It is also noticed that there is no carry as or logic does not have it and stays within the 8 bits.



AND Waveform

AND Observation:

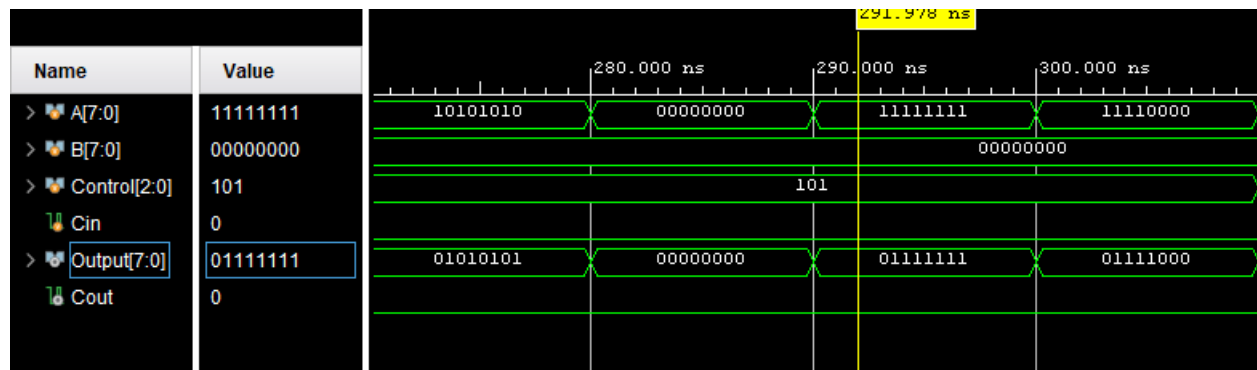
- The AND operation works in the sense that both inputs (bits) have to have the same 0 or 1 to accept. When I tested the case of alternating 1 and 0 the output is all 0 since there are no same bit values. However, when comparing the 11111111 and 01010101 the output takes the shared 1s resulting in 01010101.



SHL Waveform

SHL Observation:

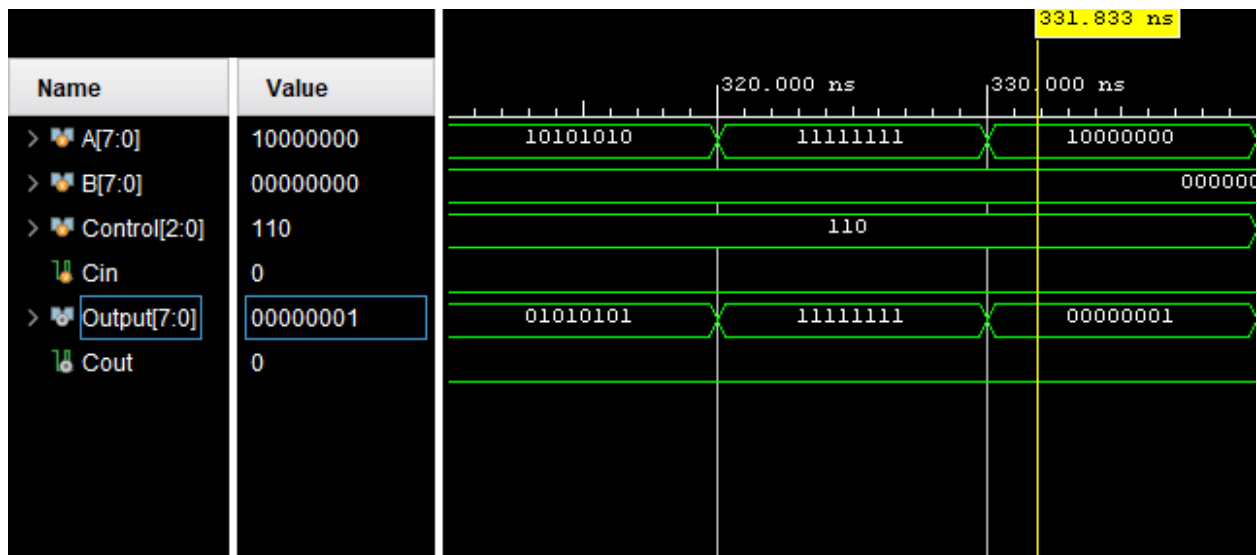
- The shift left function shifts the whole input to the left. Within the operation we only consider input A and no carry. The operation moved bits left by one position and filled the LSB with 0. A simple example is input A=11111111 and the output=11111110



SHR Waveform

SHR Observation:

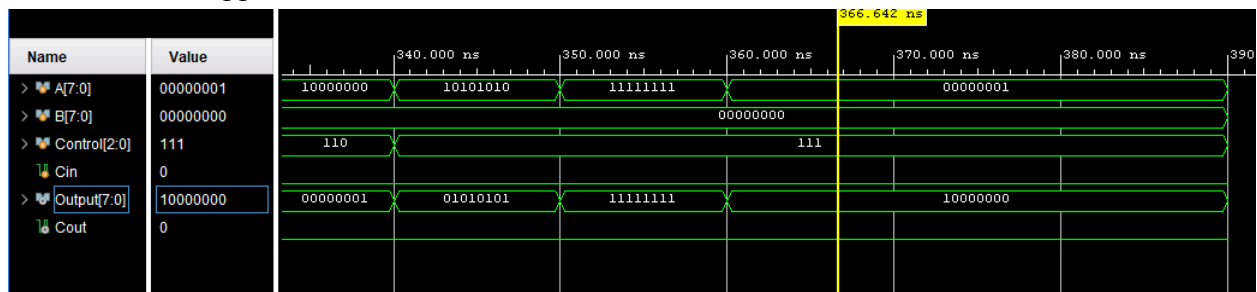
- The shift right function shifts the whole input to the right. Within the operation we only consider input A and no carry. The operation moved bits left by one position and filled the MSB with 0. An easy example is input A=11111111 and the output=01111111



ROL Waveform

ROL Observation:

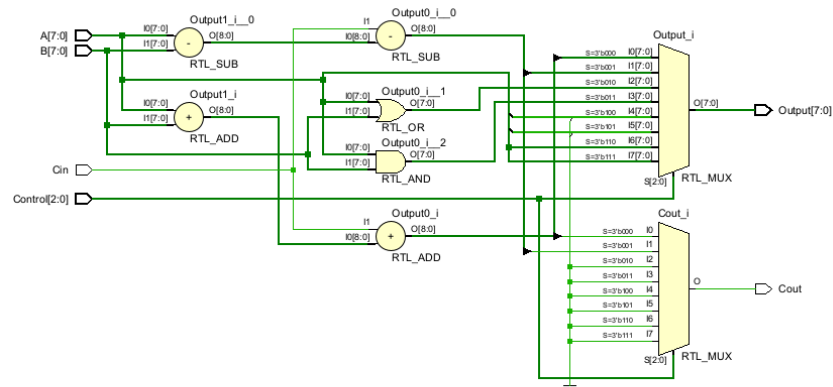
- The rotate left function saves the bits of the input but wraps bit 7 to bit 0. When testing we easily see input A=10000000 with output=00000001. This output correctly shows the MSB wrapped to LSB.



ROR Waveform

ROR Observation:

- The rotate right function saves the bits of the input but wraps bit 0 to bit 7. When testing we easily see input A=00000001 with output=10000000. This output correctly shows the LSB wrapped to MSB.



RTL Schematic

Conclusion:

- In today's lab, I learned about the arithmetic logic unit and its function. To me it seemed like a calculator where we give our inputs and select a math operation and are given the output. Although with this lab we use 8-bit inputs and combinational logic. I got to test specific cases as well as edge cases to see how the ALU would display the result. The simulation had been correct and all cases worked as they should. I did make a default for the cases in case an error occurred that would output the 8-bit unknown 'xxxxxxxx' so I could clearly see a mistake. This did not happen and all cases were properly tested with various inputs. When looking at the waveform I was able to see how carry in or logic compared and if the expected result matched the given one. Overall, I found the out of bounds cases interesting in some cases how it wraps back from the max $2^8=256$.

Appendix

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 02/11/2026 02:40:10 PM
// Design Name:
// Module Name: alu_8bit
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
// Matthew Bernardino
// ECPE 136
// Professor Anshul
// Description: Arithmetic logic unit that does math of 8 different operations, based on a 3 bit control
signal
// control signal indicates operation

module alu_8bit(
// initialize given variables
    input logic [7:0] A, // 8 bit inputs A and B
    input logic [7:0] B,
    input logic Cin, // 1 bit, carry or no carry
    input logic [2:0] Control, // 3 bit control opcode
    output logic [7:0] Output, // 8 bit output
    output logic Cout // 1 bit, carry or no carry
);

    // need to create internal signals that are 9 bits, adding 8 bit numbers can result in 9 bits with a carry
    logic [8:0] sum_add;
    logic [8:0] sum_sub;

    // use always_comb for combinational logic
    // make switch statement to check with new inputs always changing
    always_comb begin
        sum_add = A + B + Cin; // addition formula, if carry then 9 bits

        sum_sub = A - B - Cin; // subtraction formula, if borrow bit
```

```

// create default values, this prevents numbers from being remembered
Output = 8'b0; // default output to 0 every new arithmetic
Cout = 1'b0; // onlt a 1 or 0 for carry

// use case statements for each arithemsitc operation
case (Control)
  3'b000: begin // add operation
    Output = sum_add[7:0]; // take first 8 bits as addition result
    Cout = sum_add[8]; // carry 1 as 9th bit if there is one like edge cases
  end

  3'b001: begin // sub operation
    Output = sum_sub[7:0]; // take first 8 bits as subtraction result
    Cout = sum_sub[8]; // borrow out 1 as 9th bit if there is a negative result
  end

  3'b010: begin // OR operation
    Output = A | B; // Follows OR logic of 1 being dominant output.
    Cout = 1'b0; // no carry in this logic
  end

  3'b011: begin // AND operation
    Output = A & B; // both inputs must be the same
    Cout = 1'b0; // no carry in this logic
  end

  3'b100: begin // SHL operation
    Output = {A[6:0], 1'b0}; // shift left logic, take the 7 bits and shift them left, LSB = 0 // 8th
bit gets dropped
    Cout = 1'b0; // no carry in this logic
  end

  3'b101: begin // SHR operation
    Output = {1'b0, A[7:1]}; // shift right logic, take the 7 bits and shift them right, MSB = 0 //
0th bit gets dropped
    Cout = 1'b0; // no carry in this logic
  end

  3'b110: begin // ROL operation
    Output = {A[6:0], A[7]}; // Rotate left wraps the A[7] bit around to A[0], no bit loss
    Cout = 1'b0; // no carry in this logic
  end

  3'b111: begin // ROR operation
    Output = {A[0], A[7:1]}; // Rotate right wraps the A[0] bit around to A[7], no bit loss
    Cout = 1'b0; // no carry in this logic
  end

// need default for case statement for any non control values, error catching

```

```

        default: begin
            Output = 8'bx; // sets all 8 bits to x as unknown, easy tell for error
            Cout = 1'b0; // set carry out to 0, not used anyways
        end

    endcase

end

endmodule

```

alu_8bit.sv

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 02/11/2026 02:40:29 PM
// Design Name:
// Module Name: alu_8bit_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
// Matthew Bernardino
// ECPE 136
// Professor Anshul
// Description: test bench to test inputs A and B as well as Control selecting each opcode

module alu_8bit_tb();
    // Variable setup input and output
    reg [7:0] A;
    reg [7:0] B;
    reg [2:0] Control;
    reg Cin;
    wire [7:0] Output;
    wire Cout;

    ///instantiate the design module and connect the variable declared above

```

```

alu_8bit alu_inst(.A(A), .B(B), .Control(Control), .Cin(Cin), .Output(Output), .Cout(Cout));

// Edge case testing
initial begin

// start all inputs at 0
    A = 8'h00;
    B = 8'h00;
    Cin = 1'b0;
    Control = 3'b000;

    #10; //delay for operation start

// ADD = Control 000
    // simple add 2 + 3
    Control = 3'b000;
    A = 8'h02;
    B = 8'h03;
    Cin = 1'b0;
    #10; //delay until next inputs

    // focus on A being the FF or 01
    // edge case 0 + 15
    A = 8'h00;
    B = 8'h0F;
    Cin = 1'b0;
    #10;

    // edge case FF 255 + 1, should show cout = 1 output = 0
    A = 8'hFF;
    B = 8'h01;
    Cin = 1'b0;
    #10;

    // focus on B being the FF or 01
    // edge case 0 + 15
    B = 8'h00;
    A = 8'h0F;
    Cin = 1'b0;
    #10;

    // edge case FF 255 + 1, should show cout = 1 output = 0
    B = 8'hFF;
    A = 8'h01;
    Cin = 1'b0;
    #10;

    // add a and b and add in carry in 1 + 3 + 1 = 5
    A = 8'h01;
    B = 8'h03;

```

```
Cin = 1'b1;  
#10;
```

```
// add a and b and add in carry in  $255 + 255 + 1 = 511$  , carry out 1, overflow case,  
A = 8'hFF;  
B = 8'hFF;  
Cin = 1'b1;  
#10;
```

```
// Subtraction Cases Control: 001
```

```
// simple add  $2 - 3 = 1$ 
```

```
Control = 3'b001;  
A = 8'h03;  
B = 8'h02;  
Cin = 1'b0;  
#10; //delay until next inputs
```

```
// focus on A being the FF or 01
```

```
// edge case  $0 - 5 = -5$  have Cout = 1 for borrow/ - indicator, goes to 251
```

```
A = 8'h00;  
B = 8'h05;  
Cin = 1'b0;  
#10;
```

```
// edge case FF  $255 - 1 = 254$ , should show cout = 0
```

```
A = 8'hFF;  
B = 8'h01;  
Cin = 1'b0;  
#10;
```

```
// focus on B being the FF or 01
```

```
// edge case  $10 - 0 = 10$  0x0A
```

```
A = 8'h0A;  
B = 8'h00;  
Cin = 1'b0;  
#10;
```

```
// edge case FF  $5 - 255 = -250$ , should show cout = 1 borrow output = 6 (0x06) wraps
```

```
A = 8'h05;  
B = 8'hFF;  
Cin = 1'b0;  
#10;
```

```
// sub  $3 - 1 - 1 = 1$ , carry in adds extra -1
```

```
A = 8'h03;  
B = 8'h01;  
Cin = 1'b1;  
#10;
```

```
// overflow: sub  $0 - 1 - 1 = -2 = 254$  0xFE with Cout = 1 borrow for negative result
```

```
A = 8'h00;  
B = 8'h01;  
Cin = 1'b1;  
#10;
```

// OR Operation Examples Control: 010

```
Control = 3'b010;  
A = 8'hAA; // 10101010 or 01010101 = 11111111 0xFF  
B = 8'h55;  
Cin = 1'b0;  
#10; //delay until next inputs
```

```
A = 8'h00; // 00000000 or 11110000 = 11110000 0xF0  
B = 8'hF0;  
#10; //delay until next inputs
```

```
A = 8'hFF; // 11111111 or 00001111 = 11111111 0xFF  
B = 8'h0F;  
#10; //delay until next inputs
```

```
A = 8'h00; // 00000000 or 010101010 = 010101010 0xAA  
B = 8'hAA;  
#10; //delay until next inputs
```

```
A = 8'h55; // 01010101 or 10101010 = 11111111 0xFF  
B = 8'hAA;  
Cin = 1'b0;  
#10; //delay until next inputs
```

// AND Operation Examples Control: 011

```
Control = 3'b011;  
A = 8'hAA; // 10101010 and 01010101 = 00000000 0x00  
B = 8'h55;  
Cin = 1'b0;  
#10; //delay until next inputs
```

```
A = 8'hF0; // 11110000 and 10101010 = 10100000 0xA0  
B = 8'hAA;  
#10; //delay until next inputs
```

```
A = 8'h55; // 01010101 and 11111111 = 01010101 0x55  
B = 8'hFF;  
#10; //delay until next inputs
```

// SHL Operation Examples Control: 100 // only take one input to shift

```
Control = 3'b100;  
A = 8'hAA; // (170) 10101010 shift left A = 01010100 (84)  
B = 8'h00; // stays 0 not used only input A  
Cin = 1'b0;  
#10; //delay until next inputs
```

```
A = 8'h00; // 00000000 shift left = 00000000, sanity check
#10;
```

```
A = 8'hFF; // 11111111 shift left = 11111110
#10;
```

```
A = 8'hF0; // 11110000 shift left = 11100000 //
#10;
```

```
// SHR Operation Examples Control: 101 // only take one input to shift
```

```
Control = 3'b101;
```

```
A = 8'hAA; // (170) 10101010 shift right A = 01010101 0x55
```

```
B = 8'h00; // stays 0 not used only input A
```

```
Cin = 1'b0;
```

```
#10; //delay until next inputs
```

```
A = 8'h00; // 00000000 shift right = 00000000, sanity check
#10;
```

```
A = 8'hFF; // 11111111 shift right = 01111111
#10;
```

```
A = 8'hF0; // 11110000 shift right = 01111000
#10;
```

```
//ROL Operation Examples Control: 110 MSB wraps to LSB
```

```
Control = 3'b110;
```

```
A = 8'hAA; //10101010 rotate left = 01010101 0x55
```

```
B = 8'h00; // stays 0 not used only input A
```

```
Cin = 1'b0;
```

```
#10; //delay until next inputs
```

```
A = 8'hFF; // 11111111 rotate left = 11111111 same
#10; //delay until next inputs
```

```
A = 8'h80; // 10000000 rotate left = 00000001
#10; //delay until next inputs
```

```
//ROR Operation Examples Control: 111 LSB wraps to MSB
```

```
Control = 3'b111;
```

```
A = 8'hAA; // 10101010 rotate right = 01010101 0x55
```

```
B = 8'h00; // stays 0 not used only input A
```

```
Cin = 1'b0;
```

```
#10; //delay until next inputs
```

```
A = 8'hFF; // 11111111 rotate right = 11111111 same
#10; //delay until next inputs
```

```
A = 8'h01; // 00000001 rotate right = 10000000 = 0x80
```

```
#10; //delay until next inputs

#20 $finish;

end

// Monitor output
always @(A, B, Cin, Control, Output, Cout)
    $monitor("Time=%0t ns, Control=%b, A=0x%h, B=0x%h, Cin=%b, Output=0x%h, Cout=%b",
$time, Control, A, B, Cin, Output, Cout);

endmodule
```

alu_8bit_tb.sv